

# Project 1 — The Arcade

*HTML, CSS, and Design*

## Overview

This first assignment has you building the site that every other project this semester will be added to. By the end of the course it will hold four games; right now it holds one puzzle, plus enough about you that the link is worth putting on a resume.

You will build a small static site containing a landing page, a playable mini crossword, a page about your experience and education, and a page describing how to contact you. The crossword is the centerpiece and the reason this project is more than a personal site — you are going to build a real interactive grid using nothing but HTML and CSS (no JavaScript!). For those unfamiliar, here is an example of a crossword: <https://www.boatloadpuzzles.com/playcrossword> (yours will be much smaller and simpler).

Keep in mind who this is for. You will be sharing this URL with recruiters and colleagues, and you will be adding to it for the next three months. Build something you would be happy for someone to find under your name.

This project must be completed solo. As a reminder, you should only use AI to provide hints, suggestions or to help debug: the code should be produced entirely by you.

## Restrictions

No JavaScript at all — no imported JS libraries, no Ruby, no other programming language, no logic of any kind on the page. Everything interactive in this project has to be done with HTML and CSS alone, which is the whole exercise.

No styling libraries either: no Bootstrap, no Tailwind, no Material, no component frameworks. You will lose points for these. The goal is for you to become comfortable writing CSS yourself before you let a tool write it for you.

You MAY import font files and icon files. You may use SVG. That is the extent of what may come from outside your own repository.

## Rubric

| Item                                              | Percent |
|---------------------------------------------------|---------|
| Working GitHub repo, code quality, and a live URL | 10      |
| Four unique pages and file organization           | 10      |
| Navbar                                            | 15      |
| The puzzle                                        | 15      |
| Mobile friendly design                            | 10      |
| Use of certain HTML elements                      | 10      |
| Use of certain CSS properties and features        | 10      |

| Item                         | Percent  |
|------------------------------|----------|
| Accessibility                | 10       |
| Good and professional design | 10       |
| Writeup                      | Required |
| Extra credit                 | 5 pts    |

## Working GitHub Repo, Code Quality, and a Live URL

All of your code must be pushed, and the site must actually work for a visitor. Code should be clean, readable, and consistently formatted.

- Organize files into sensible folders. Shared CSS in a common location, page-specific CSS alongside its page
- Meaningful, consistent names for ids and classes
- Include a .gitignore (even if it may be empty for this assignment)
- No JavaScript files anywhere in the repository

## Four Unique Pages

Your site needs at least four pages:

- Landing page. It introduces the site and links to the puzzle via the header. This is your root (/)
- The puzzle page, covered in detail below (/game or /game.html)
- About you: your experience, education, and anything else professional you want visible (/about)
- Contact — how to reach you and where to find you online. Include what applies: email, GitHub, LinkedIn, and any social media links whose content you are comfortable with professionals seeing (/contact)

You are welcome to add other pages if you want.

Content is not graded on typos or grammar, and if you would rather not publish real personal details you may make them up.

## Navbar

Every page needs the same navigation (this will be copy and pasted on each page, but we'll improve this in the future)

- It stays in position no matter where the user has scrolled
- It links to every page on the site
- The current page is visually indicated, so a visitor always knows where they are
- It includes a title referencing your name
- It does not cover content unless the user has scrolled
- It is marked up as a nav containing a list of links, not a row of divs
- It is fully usable with the keyboard alone, with a visible focus indicator

## The Puzzle

A mini crossword!

### Requirements:

- A grid of at least 5×5 (can be bigger if you wish)
- Built with CSS Grid
- Blocked-out squares, visually distinct from open ones
- Clue numbers in the corner of the appropriate squares
- Across and Down clue lists, marked up as real lists
- Typeable cells — a visitor can click or tab into a square and enter a letter
- A way to reveal the solution that works for keyboard and touch users

### Hint for making the game:

An input with `maxlength="1"` in each open square gives you a working puzzle easily. It accepts a letter, it tabs to the next square, and it requires no scripting at all.

```
<input type="text" maxlength="1" aria-label="Row 1, column 2, 3 Across">
```

Every input needs an accessible name. A grid of twenty-five unlabelled text boxes is unusable with a screen reader, and solving it properly is part of the assignment.

### Hint for making the solution:

Do not gate the solution behind `:hover` alone. `Hover` does not exist on a touchscreen and cannot be reached from the keyboard, so a `hover-only` reveal is invisible to phone users and keyboard users both. Use one of these instead:

- A details / summary element, which is semantic and keyboard-accessible with no tricks
- Or build a CSS-only toggle of your own. Everything you need is in the Week 2 selector material

You are welcome to add hover effects on top of that as decoration. Just do not make them the only way in.

You may write your own mini puzzle or adapt a public-domain one or use AI to generate the clues/solutions. If you adapt one, credit it in your writeup.

## Mobile Friendly Design

The site must work as well on a phone as on a monitor. Assume the person deciding whether to interview you is looking at it on a phone.

- Everything reachable on desktop must be reachable on mobile
- Text, images, and the puzzle grid scale sensibly
- The navbar likely needs to change shape on a small screen: moving it to the bottom is one good option
- Tap targets are at least 44×44 pixels, including the puzzle squares
- We will be testing on a screen width of 320px to represent a phone and then 864px to represent a desktop

## Use of Certain HTML Elements

Your site must use each of the following at least once:

- header, nav, main, footer
- section or article
- h1 through h6, used as a real outline — never chosen because you liked the size
- a, img (with meaningful alt text), p
- ul or ol
- label and input

Your HTML must pass the W3C validator with no errors. Semantic elements are expected wherever one applies; div and span should be used sparingly.

## Use of Certain CSS Properties and Features

Your stylesheets must use each of the following at least once:

- font-family
- background or background-color
- margin and padding
- position (your sticky navbar counts)
- display: grid, the puzzle
- display: flex
- @media queries
- Custom properties (CSS variables) for your colors and spacing, defined once at the top
- At least two different pseudo-classes
- At least one pseudo-element
- :focus-visible, styled deliberately
- transition or transform

## Accessibility

- The entire site is usable with a keyboard alone, and focus is always visible
- All text meets a contrast ratio of at least 4.5:1
- Every image has appropriate alt text, and decorative images use an empty alt
- Every input has an accessible name
- No clickable divs. If it does something, it is a button or a link
- Lighthouse accessibility score of 80 or higher

Run Lighthouse before you submit. It takes ten seconds and catches most of this.

## Good and Professional Design

The site should be easy to navigate and interesting enough to hold attention.

- Decide on a type scale, a neutral color ramp, one accent color, and a spacing scale before you write CSS. Define them as custom properties
- Cap your body text at a readable line length
- Consider importing a font. Which typefaces do you actually find readable on a screen?
- Think about branding — SVG icons and a consistent visual identity go a long way
- Use spacing to group related things. Whitespace is doing work
- You should be able to justify every color and size on the page

This is not a design course, and it is not graded like one. The intention is that you can produce something worth sharing with recruiters and colleagues and that you begin to identify basic style strategies and theories.

### **Extra Credit**

Submit 48 hours before the deadline and you earn 5 extra credit points.

### **Deliverables**

Submit the following:

- A link to your live website
- A link to your GitHub repository. If it is private, add the TAs as collaborators
- A writeup answering the questions below, at least three sentences per bullet (where applicable)
- A screenshot of your Lighthouse score

#### **Write Up questions:**

- What was the most challenging part of this assignment? Did you find HTML and CSS easy or difficult to work with?
- How did you build the puzzle grid, and what other options did you consider?
- What did you take into account when designing the site? Is there anything you are particularly proud of?
- Given more time or resources, what would you add?
- How many hours did you spend on this assignment? (obviously just a sentence is sufficient)
- (Optional) If you made any assumptions about this assignment, what were they?
- If you used code or design from anywhere online (including AI), say so here. If you imported a font or icon library, or adapted an existing puzzle, note that as well